BULK CERTIFICATE GENERATOR

Technical Documentation & Submission Guide

Project Type	FastAPI REST API
Core Purpose	Generate individual PDF certificates from a bulk recipient list
Database	SQLite with SQLAlchemy
PDF Engine	ReportLab
Testing	pytest

Prepared in a clean, structured format for company/project submission. This version reorganizes the supplied documentation into a more readable technical report while retaining the project's original scope, terminology, endpoints, setup steps, design decisions, testing, and production recommendations.

1. Project Overview

Bulk Certificate Generator is a FastAPI-based REST API that accepts a bulk list of recipients and generates individual PDF certificates from a predefined template. The application tracks job progress, stores certificate status in a relational database, and allows generated certificates to be retrieved individually.

Key Capabilities

- Bulk certificate generation from a recipient list.
- Request validation using Pydantic.
- REST API implementation using FastAPI.
- SQLite relational database access through SQLAlchemy.
- Background processing for certificate generation.
- Job status and progress tracking.
- Individual certificate status tracking.
- Independent failure handling so one failed certificate does not stop the remaining certificates.
- Generated PDF certificate retrieval.

- Automated testing with pytest.
- Interactive API documentation through Swagger UI.

2. Technology Stack

Technology	Purpose
Python 3	Application programming language
FastAPI	REST API framework
SQLAlchemy	ORM and database access layer
SQLite	Relational database
Pydantic	Request/input validation
ReportLab	PDF certificate generation
pytest	Automated testing
Uvicorn	Development server

3. Project Structure

```
bulk-certificate-generator/ ■■■ app/ ■ ■■■ main.py ■ ■■■ database.py ■ ■■■ models.py ■ ■■■ schemas.py ■ ■■■ generator.py ■ ■■■ worker.py ■■■ generated/ ■■■ test_api.py ■■■ README.md ■■■ requirements.txt ■■■ .gitignore ■■■ certificates.db
```

Submission note: The supplied documentation states that **venv/**, **generated/**, **__pycache__/**, **.pytest_cache/**, and **certificates.db** should not be committed to Git.

4. Setup & Execution

Step 1 — Create and activate the virtual environment

```
python -m venv venv venv\Scripts\activate
```

Step 2 — Install dependencies

```
pip install -r requirements.txt
```

Step 3 — Start the application

```
uvicorn app.main:app --reload
```

Step 4 — Open the API

```
http://127.0.0.1:8000
```

Swagger / OpenAPI documentation

```
http://127.0.0.1:8000/docs
```

5. REST API Reference

Operation	Method	Endpoint	Purpose
Create bulk job	POST	/jobs	Create a certificate generation job
Check job	GET	/jobs/{job_id}	View job status and progress
List certificates	GET	/jobs/{job_id}/certificates	View certificates belonging to a job

Retrieve certificate	GET	/certificates/{certificate_id}	Return the generated PDF file

Create a Bulk Certificate Job

```
POST /jobs { "event_name": "Python Workshop 2026", "certificate_date": "2026-10-08", "recipients": [
{"name": "Madhuri", "email": "madhuri@example.com"}, {"name": "Rahul", "email": "rahul@example.com"}
] }
```

Example response:

```
{ "job_id": 1, "status": "PENDING", "total": 2 }
```

The endpoint returns HTTP 202 Accepted because certificate generation is processed as a background task.

Check Job Status

```
GET /jobs/{job_id} { "job_id": 1, "status": "COMPLETED", "total": 2, "successful": 2, "failed": 0,
"progress": "100%" }
```

Supported job statuses:

- **PENDING**
- **PROCESSING**
- **COMPLETED**
- **COMPLETED_WITH_ERRORS**

List Certificates for a Job

```
GET /jobs/{job_id}/certificates [ { "certificate_id": 1, "recipient_name": "Madhuri",
"recipient_email": "madhuri@example.com", "status": "SUCCESS", "file_path":
"generated/certificate_1.pdf", "error_message": null } ]
```

Retrieve an Individual Certificate

```
GET /certificates/{certificate_id}
```

If the certificate was generated successfully, the API returns the generated PDF file.

6. Validation & Certificate Generation

Input Validation

The API validates incoming recipient data before creating a job. Invalid examples include:

- Invalid email address.
- Missing recipient name.
- Missing email address.
- Empty recipient list.

Invalid requests are rejected by FastAPI/Pydantic with HTTP 422 Unprocessable Entity.

Certificate Generation

Certificates are generated using ReportLab and a predefined certificate template containing:

- Certificate title
- Recipient name
- Event name
- Certificate date
- Certificate ID

Each generated certificate is saved using the following naming convention:

```
generated/certificate_<certificate_id>.pdf
```

7. Bulk Processing & Failure Handling

Each certificate is processed independently. The worker wraps individual certificate generation in a try/except block. If one certificate fails, the system records the failure and continues processing the remaining recipients.

Failure handling sequence

- The certificate is marked as **FAILED**.
- The error message is stored in the database.
- The job's failed counter is increased.
- Processing continues for the remaining recipients.

Example: if 10 certificates are requested and 1 fails, the job can finish with 9 successful certificates, 1 failed certificate, 100% progress, and status **COMPLETED_WITH_ERRORS**.

```
Total: 10 Successful: 9 Failed: 1 Progress: 100% Status: COMPLETED_WITH_ERRORS
```

8. Database Design

The application uses two main database tables.

Table	Stored information
Job	id, status, total, successful, failed, created_at
Certificate	id, job_id, recipient_name, recipient_email, status, file_path, error_message

```
Job ■■■ Certificate ■■■ Certificate ■■■ Certificate One job can contain many certificates.
```

9. Progress Tracking & Background Processing

Progress Calculation

```
processed = successful + failed progress = (processed / total) * 100
```

The database is updated after each certificate so the current state of the job can be queried.

Background Processing Flow

```
Client ■ ■ POST /jobs ▼ FastAPI ■ ■ Create Job + Certificates ▼ Database ■ ■ Start background task ▼ Worker ■■■ Certificate 1 → SUCCESS ■■■ Certificate 2 → SUCCESS ■■■ Certificate 3 → FAILED ■ ▼ Database updated
```

FastAPI BackgroundTasks is used to process certificate generation after the API accepts the request. For a production system with a large number of certificates, a durable task queue such as Celery/RQ with Redis would be a stronger choice.

10. Testing

The project includes automated tests covering the main assignment requirements:

- Job creation
- Input validation
- Certificate generation
- Job progress

- Individual certificate failure

Run the tests with

```
pytest test_api.py -v
```

Expected result:

```
5 passed
```

If a direct certificate download/retrieval test has also been added, the total number of passing tests may be higher.

11. Design Decisions

Why FastAPI?

Simple REST API development; automatic request validation; automatic Swagger/OpenAPI documentation; easy background task support; good developer experience.

Why SQLite?

A relational database that is simple to set up and does not require a separate database server. It is suitable for the assignment. PostgreSQL would be stronger for production.

Why SQLAlchemy?

Provides a clean ORM layer and keeps database operations separate from API logic.

Why BackgroundTasks?

Keeps the API responsive while certificate files are generated and is sufficient for the scope of the assignment. A dedicated task queue would be preferred for high-volume production workloads.

12. End-to-End Workflow

1. Client sends a bulk recipient list to POST /jobs.

2. FastAPI validates the request.

3. A job is created in the database.

4. Certificate records are created for each recipient.

5. A background task starts processing the job.

6. Each certificate is generated independently.

7. Successful certificates are saved as PDF files.

8. Failed certificates are marked as FAILED with an error message.

9. Job progress is updated after each certificate.

10. Client can check job status using GET /jobs/{job_id}.

11. Client can list generated certificates using GET /jobs/{job_id}/certificates.

12. Client can retrieve an individual PDF using GET /certificates/{certificate_id}.

13. Production Improvements

For a production deployment, the supplied documentation recommends considering the following improvements:

- PostgreSQL instead of SQLite.
- Celery/RQ with Redis for durable background jobs.

- Object storage such as S3 for generated certificates.
- Authentication and authorization.
- Rate limiting.
- Structured logging.
- Better HTTP error responses.
- Job cancellation and retry support.
- Database indexes for large datasets.
- Docker/container deployment.
- Monitoring and health checks.
- More comprehensive integration tests.

14. Company Submission Checklist

Before submitting the project, verify that the repository contains:

- **app/**
- **test_api.py**
- **README.md**
- **requirements.txt**
- **.gitignore**

Do not submit

- **venv/**
- **__pycache__/**
- **.pytest_cache/**
- **generated/**
- **certificates.db**

Final verification commands

```
pytest test_api.py -v uvicorn app.main:app --reload
```

Then open:

```
http://127.0.0.1:8000/docs
```

Verify that the API is running correctly and that the Swagger documentation is accessible before submission.

15. Conclusion

This project demonstrates a complete bulk certificate generation workflow using FastAPI, SQLAlchemy, SQLite, Pydantic, and ReportLab. It supports input validation, background processing, progress tracking, independent failure handling, PDF generation, certificate retrieval, and automated testing.

Submission-ready summary: The implementation provides a clear separation between API handling, validation, database operations, certificate generation, and background processing. The documented production improvements also provide a practical path for scaling the solution beyond the assignment scope.